

How the code is built

Read this after you have walked October 1 on Staging. The screens will give these files a place to land. You will not change code on your first day.

What kind of program this is

In school you may have used a framework that generates folders called models, views, and controllers, or a JavaScript app that runs in the browser and talks to an API. This project is older in style, and that is a decision, not a backlog item.

It is PHP pages on a server, a MariaDB database, and ordinary CSS and JavaScript. There is no React app, no Composer project, and no PHPUnit. A rewrite into a framework is not the work. The work is that one bakery day stays trustworthy.

When the code looks uneven, assume it got that way by closing a real handoff. Understand the handoff before you clean the file.

One web address, one file

The folder you will open is `bakery/`. A file in that top folder is a page. `daily_orders.php` is the Daily Orders address you already opened. Shared rules live in `includes/`, in functions whose names start with `bakery_`. Shared page chrome is the header, the navigation, and the footer.

Place	What you will find
*.php in the top folder	One URL. Check who may open it, validate the form, call a function, print the page.
includes/*.php	The business rule, once. Orders, inventory, billing, and delivery confirmation live here.
css/ and includes/*.js	Look and behavior in the browser. No bundler.
lang/en.php and lang/es.php	Every phrase a person sees, in both languages, with the same keys.
database/schema/	Numbered SQL files that add tables and columns. Applied in order. Never renamed after they have been used.
tests/run_*.php	Scripts that call those functions and check the database. This is the test suite.

What happens when you open a page

1. The page loads configuration: environment, session, the Los Angeles clock, and the language.
2. It opens the database and checks login, role, and, on any save, a token that proves the form came from our page.
3. If you submitted a form, the page calls one function in `includes/`. It should not invent a second copy of the same SQL.
4. It prints the header, the page, and the footer.

A useful exercise, after your walk: pick the delivered stop you opened on My Route. In the repository, search for `bakery_confirm_delivery`. That function is the moment a delivery becomes a bill. Read it. You are not asked to change it.

The data, only as far as the day requires

Customers are the identity. A wholesale store, a portal login, and a student in the baking classes all point at a customer row. New side products get their own tables. They do not keep adding columns to the order tables.

- **Standing orders** are the weekly habit.
- **Daily orders and their lines** are one date's promise.
- **Assignments** are the route: which driver, which position, delivered or not.
- **A production commit** is the bake list the baker executes.
- **Inventory movements** are an append-only story of bread: made, loaded, delivered, returned, wasted.
- **Snapshot columns on the order** are the prices and total frozen at delivery.

If you need "how much bread is expected that day," use the demand helpers. Do not add up the raw tables yourself. You will drop pauses, inactive stores, or the rule that one store's special order does not speak for every store.

Rules that matter when you eventually change code

You saw these on the screens. They are also rules for a patch.

1. Dated beats standing, per customer. Never for the whole date at once.
2. Editing the habit does not rewrite a past date. Editing a date does not rewrite the habit.
3. Generating the date again must keep quantities a person already changed.
4. Confirming a delivery is one transaction: pieces, credits, prices, and cash together.
5. A historical invoice never looks up today's catalog price.

6. The order status and the route-stop status are two fields. They move on different writes and have to stay in agreement.
7. Loading the van moves bread onto the van. It does not record the sale. The route closeout does that math.
8. Marking an exception “handled” must not hide the fact that it is still true.

The long version is section 4 of `BAKERY_PRODUCT_CONTEXT.md`. Read that section before you change orders, prices, inventory, or invoices. You do not need it to finish the Staging walk.

Three databases on the laptop, and two on the host

You will not set the laptop up on day one. When you do, ask which database a command will write to before you run it. The names look similar. They are not interchangeable.

Name	What it is	May you write?
bakerysf_local	A nightly mirror of production, for looking.	No.
bakerysf_stage_local	Everyday development on the laptop.	Yes, once you are set up.
bakerysf_test	A disposable database the tests wipe.	Only the test scripts.
bakerysoftware	Staging, the site in this packet.	Only by using the site, and only when Danny asks you to save something.
bakerysf	Live. Today’s bread and invoices.	No.

Git is not the website

There are three copies of the *code*, and updating one does not update the next.

1. **GitHub** (`dgabriner/bakery`, branch `main`) is the history of the code. A commit does not change what a phone is running.
2. **Staging** (<https://staging.sourflour.org/>) is the copy you just used. It is updated by a separate file sync.
3. **Live** (<https://bakery.sourflour.org/bake/>) is promoted by Danny from a screen called Staging Manager. You do not copy files there, and you do not import a database over it.

Passwords, `.env` files, uploads, and database dumps do not go in Git.

How a small change is supposed to go

When Danny asks you to change something, this is the shape of a good change. It is not a request to start one now.

1. Name the date, the person, and the screen where that person already decides.
2. Click that screen on Staging, and the screen before it and the screen after it.
3. Change the function that already owns that fact. Prefer the existing page over a new page.
4. If a person will see a new sentence, add it in English and in Spanish in the same change.
Spanish is a translation, not a copy of the English.
5. Run the test script that covers those files, on the disposable test database.
6. Say what you did not understand. Leave Live alone.

A small change that finishes a handoff is the sophisticated one. A new home page, however clean, is a failed change. This app already has too many screens.

What not to do in the first month

- Do not propose replacing PHP with a framework as your first idea.
- Do not add a new top-level page because a menu feels missing.
- Do not build a second way to confirm a delivery or a second way to number an invoice.
- Do not price an old invoice from the current product price.
- Do not run a command whose name says production, live, or push until you can say which database it writes.
- Do not put customer names, codes, or dumps into Git or into a chat with an outside model.

If you want the long version later

Read	When
BAKERY_PRODUCT_CONTEXT.md	Before you change a feature. Sections 1 through 5, and section 4 as the rule book.
ARCHITECTURE.md	When you need the request and the folder map in more detail.
docs/DEV_WORKFLOW.md and docs/LOCAL_SETUP.md	When Danny sits with you to set up the laptop.
docs/archive/	Do not. Those notes describe old plans. They will send you to fix things that already shipped.

If a document and the code disagree, the code is what the bakery is running. Then the document should be corrected. That is a normal part of the job, and it is a good first kind of

change: small, checkable, and hard to do by accident to a store.

Sour Flour OS intern packet, October 2026. Practice on Staging. Live stays with the bakery.